

MANUAL26. УРОК 8. ТАБЛИЦА РЕШЕНИЙ: КАК ПРОВЕРЯТЬ КОМБИНАЦИИ УСЛОВИЙ БЕЗ ХАОСА.

На прошлом уроке мы научились работать с граничными значениями и увидели, почему ошибки часто живут на «краях» диапазонов. Это сильная техника для параметров с лимитами. Но в реальных задачах часто проблема не в одном параметре, а в сочетании нескольких условий. Именно здесь многие команды начинают терять контроль: комбинаций много, проверок много, логика путается. В этом уроке мы разбираем инструмент, который помогает навести порядок.

Что нового в этой теме

Тема урока — таблица решений. Это техника тест-дизайна, которая помогает

структурировать правила вида «если выполнены такие условия, то система должна дать такой результат». Она особенно полезна там, где поведение зависит сразу от нескольких флагов, ролей, статусов, ограничений и бизнес-правил.

Почему комбинации условий ломают даже аккуратные проверки?

Одно условие проверить легко. Сложность начинается, когда условий несколько: ранг, флаг безопасности, лимит слотов, статус аккаунта и так далее. Комбинации растут очень быстро, и без структуры мы почти неизбежно что-то пропускаем.

Обычно в такой ситуации происходят две вещи: часть веток вообще не проверяется, а часть проверяется по кругу под разными формулировками. То есть времени тратится много, а уверенность в покрытии все равно низкая.

Таблица решений решает именно эту проблему. Она раскладывает логику по полочкам: какие условия есть, какие у них

состояния и что система должна сделать в каждом сочетании. После этого тестирование перестает быть «проверкой на память» и становится прозрачной системой правил.

Что такое таблица решений простыми словами

Таблица решений

это таблица, где в одной части перечислены условия, а в другой - ожидаемые действия или результаты системы.

Каждая колонка таблицы — это отдельное правило (или отдельный сценарий), то есть конкретная комбинация условий.

Обычно таблица строится так:

- строки с условиями (например, «ранг достаточный?», «2FA включена?», «есть места?»);
- строки с действиями/результатами (например, «допустить к регистрации», «показать ошибку»);
- колонки с вариантами сочетаний.

Главный смысл техники: не дать системе «спрятать» неочевидную комбинацию, которая потом станет инцидентом в проде.

Когда таблица решений особенно полезна?

Техника особенно сильна там, где:

- несколько независимых условий влияют на результат;
- есть бизнес-правила с исключениями;
- одинаковые по виду сценарии дают разные результаты в зависимости от статусов;
- команде важно прозрачно объяснить логику проверок.

Типичные примеры: доступы по ролям, скидки и бонусы, правила оплаты, валидации при разных флагах, условия допуска к событию, матчмейкинг с ограничениями, модерация контента, античит-проверки.

В таких задачах таблица решений экономит время не только QA, но и всей команды: аналитики, разработчики и тестировщики начинают говорить на одном языке правил.

Rocket League:

Возьмем игровой контекст Rocket League. Допустим, в игре

запускают режим командного турнира, и система решает, можно ли зарегистрировать команду.

Условия:

1. У капитана достаточный ранг.
2. У всех участников включена двухфакторная защита.
3. В команде есть свободные слоты для регистрации.

На словах правило звучит просто. Но если разложить на комбинации, появляются нюансы:

- ранг ок, 2FA ок, слоты есть -> регистрация разрешена;
- ранг ок, 2FA нет, слоты есть -> отказ с подсказкой включить 2FA;
- ранг не ок, 2FA ок, слоты есть -> отказ по рангу;
- ранг ок, 2FA ок, слотов нет -> отказ по лимиту состава;
- и так далее.

Даже в таком небольшом примере без таблицы легко перепутать ожидаемые сообщения или забыть одну из комбинаций. Таблица решений сразу показывает полную картину.

Как строить таблицу решений пошагово

Шаг 1. Выпишите условия, которые реально влияют на результат.

Не нужно тянуть в таблицу все подряд — только те факторы, которые меняют поведение системы.

Шаг 2. Для каждого условия определите возможные значения.

Чаще всего это да/нет, но могут быть и больше вариантов (например, роль: игрок/капитан/модератор).

Шаг 3. Сформируйте комбинации условий.

На этом этапе удобно увидеть, какие сочетания теоретически возможны, а какие невозможны по логике продукта.

Шаг 4. Для каждой комбинации зафиксируйте ожидаемое действие системы.

Важно писать конкретно:
«показать ошибку X»,
«разрешить регистрацию»,
«заблокировать кнопку», а не
«ведет себя корректно».

Шаг 5. Уберите дубли и невозможные сценарии.

Иногда разные комбинации дают один и тот же результат. Их можно объединить, чтобы не раздувать набор тестов.

Шаг 6. Преобразуйте таблицу в тест-кейсы.

Каждое полезное правило в таблице должно получить проверку.

Как не утонуть в количестве комбинаций

Самая частая тревога: «если условий много, таблица будет огромной». Это правда. Но цель таблицы не в том, чтобы механически проверять абсолютно все сочетания без разбора. Цель — осознанно управлять покрытием.

Есть три рабочих приема:

1. Убирать невозможные комбинации сразу (например, система не позволяет такую конфигурацию в интерфейсе).
2. Объединять комбинации с одинаковым ожидаемым

результатом.

3. Приоритизировать правила по риску пользователя и бизнес-ценности.

Так вы получаете не «гигантскую матрицу ради матрицы», а управляемый набор проверок, который реально можно выполнить в релизное окно.

Какие дефекты хорошо ловятся таблицей решений

1. Первая группа — пропущенные ветки логики. Когда команда реализовала основное поведение, но забыла редкую комбинацию условий.
2. Вторая группа — противоречивые правила. Например, одно условие говорит «разрешить действие», другое — «запретить», и система ведет себя нестабильно.
3. Третья группа — неконсистентные сообщения и действия. Система блокирует корректно, но показывает неверный reason-код или не ту подсказку.
4. Четвертая группа — рассинхрон между слоями. UI позволяет действие при

определенной комбинации, а сервер отклоняет, потому что у него другая логика.

Ограничения техники: чего таблица решений не заменяет

Таблица решений не заменяет граничные значения, эквивалентные классы и исследовательское тестирование. Она отвечает на вопрос «что делать при этой комбинации условий», но не гарантирует, что внутри каждого условия нет собственных проблем.

Например, таблица может сказать «если ранг достаточный и 2FA включена, регистрация разрешена». Но если поле ранга ломается на границе или 2FA-проверка падает по таймауту, это уже другие техники и другой фокус.

Поэтому лучший результат дает комбинация подходов: таблица решений + проверки границ + проверки на классы + здравый exploratory.

Типичные ошибки начинающих

1. Первая ошибка — включать в таблицу условия, которые не влияют на результат. Таблица разрастается, но полезной информации не становится больше.
 2. Вторая ошибка — не фиксировать ожидаемое действие явно. Фраза «должно работать» бесполезна в таблице решений.
 3. Третья ошибка — забывать про невозможные комбинации. Их нужно отмечать отдельно, а не тратить на них время как на обычные тесты.
 4. Четвертая ошибка — не синхронизировать таблицу с требованиями после изменений. Если правило в продукте изменилось, старая таблица начинает вводить команду в заблуждение.
 5. Пятая ошибка — воспринимать таблицу как финальный артефакт без превращения в тесты. Таблица полезна только тогда, когда по ней действительно выполняются проверки.
-

Итоги занятия

На этом уроке мы разобрали, как системно тестировать сложные правила, где поведение зависит

от нескольких условий
одновременно. Таблица решений
помогает сделать проверки
прозрачными, убрать хаос и
повысить предсказуемость
покрытия.

Профессиональный вывод:
сильный QA не пытается
удержать комбинации в голове.
Он выносит логику в структуру,
которую можно проверить,
обсудить и поддерживать в
команде.

Глоссарий к занятию

1. **Таблица решений** – это структурированное представление условий и ожидаемых действий системы для разных комбинаций.
2. **Условие** – это фактор, который влияет на поведение системы в сценарии.
3. **Правило** – это конкретная комбинация условий и соответствующее ей ожидаемое действие.
4. **Невозможная комбинация** – это сочетание условий, которое не может возникнуть по логике продукта.
5. **Покрытие комбинаций** – это степень того, насколько проверки охватывают значимые сочетания условий.

